you need will vary depending on the third-party API you're calling. This example uses strongly typed configuration to inject the secret API key, as you saw in chapter 10.

Listing 33.9 Creating a custom `HttpMessageHandler`

```
public class ApiKeyMessageHandler : DelegatingHandler                ❶
{
    private readonly ExchangeRateApiSettings _settings;              ❷
    public ApiKeyMessageHandler(                                     ❷
        IOptions<ExchangeRateApiSettings> settings)                  ❷
    {                                                                ❷
        _settings = settings.Value;                                  ❷
    }                                                                ❷

    protected override async Task<HttpResponseMessage> SendAsync(    ❸
        HttpRequestMessage request,                                  ❸
        CancellationToken cancellationToken)                         ❸
    {
        request.Headers.Add("API-KEY", _settings.ApiKey);            ❹

        HttpResponseMessage response =                               ❺
            await base.SendAsync(request, cancellationToken);        ❺

        return response;                                             ❻
    }
}
```

❶ Custom HttpMessageHandlers should derive from DelegatingHandler.

❷ Injects the strongly typed configuration values using DI

❸ Overrides the SendAsync method to implement the custom behavior

❹ Adds the extra header to all outgoing requests

❺ Calls the remainder of the pipeline and receives the response

❻ You could inspect or modify the response before returning it.

To use the handler, you must register it with the DI container and add it to a named or typed client. In the following listing, we add it to the `ExchangeRatesClient`, along with the transient error handler we